

Introduction to Data Processing

Lionel Fillatre
2025-2026

lionel.fillatre@univ-cotedazur.fr

Lecture 4

Introduction to Modelling

Outlines

- Data processing pipeline
- Features engineering
- Linear regression
- Logistic regression
- Softmax regression
- Model validation
- Conclusion

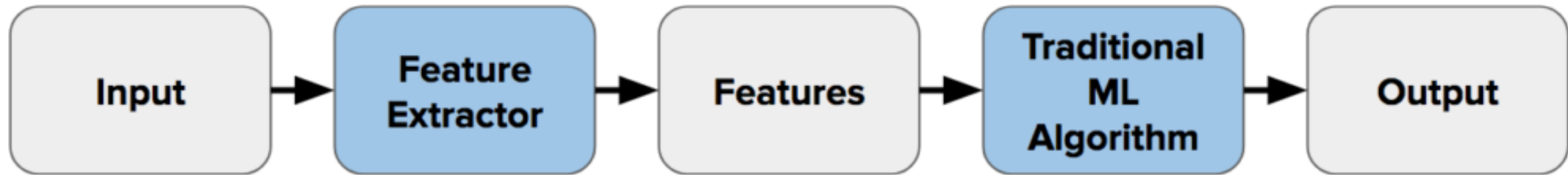
Data Processing Pipeline

Traditional Machine Learning Flow

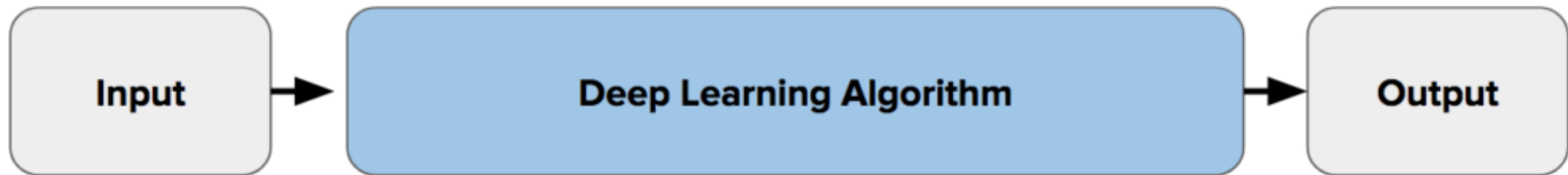


Traditional Machine Learning Flow

Deep Learning Flow

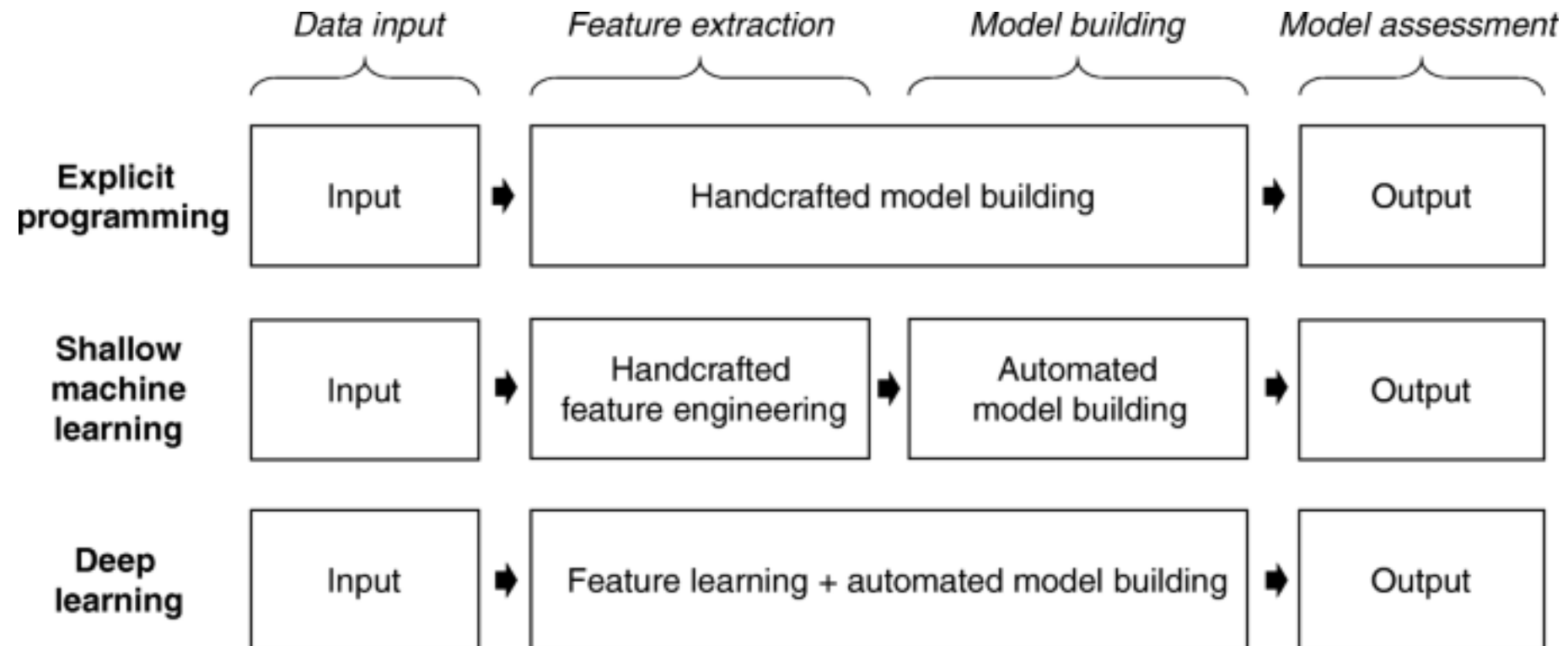


Traditional Machine Learning Flow

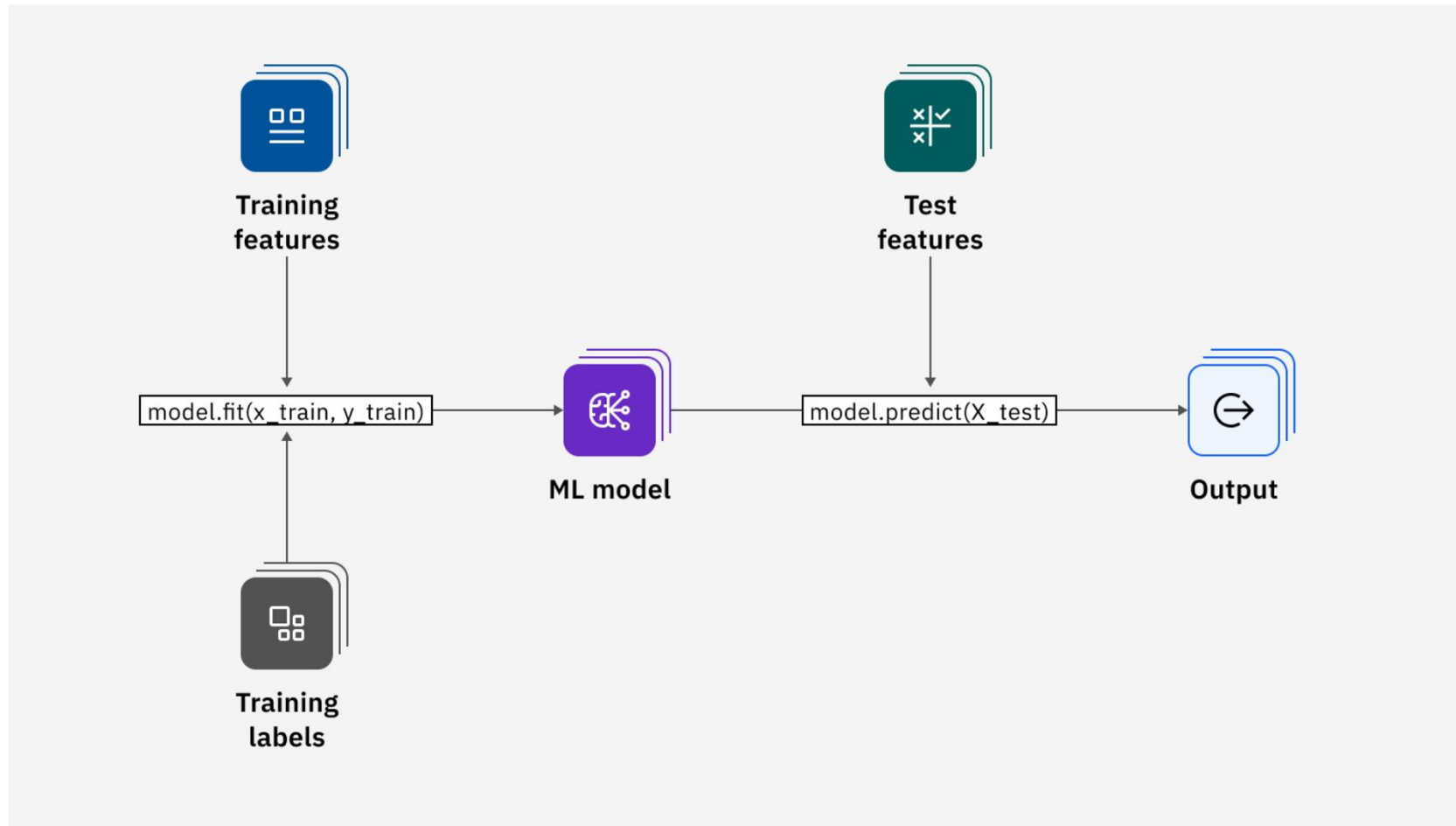


Deep Learning Flow

From explicit programming to deep learning



How to train a model?



Features Engineering

Titanic Challenge

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q

Aggregation of features

Sex	female	male
Title		
Capt	0	1
Col	0	2
Countess	1	0
Don	0	1
Dr	1	6
Jonkheer	0	1
Lady	1	0
Major	0	2
Master	0	40
Miss	182	0
Mlle	2	0
Mme	1	0
Mr	0	517
Mrs	125	0
Ms	1	0
Rev	0	6
Sir	0	1



	Title	Survived
0	Master	0.575000
1	Miss	0.702703
2	Mr	0.156673
3	Mrs	0.793651
4	Rare	0.347826

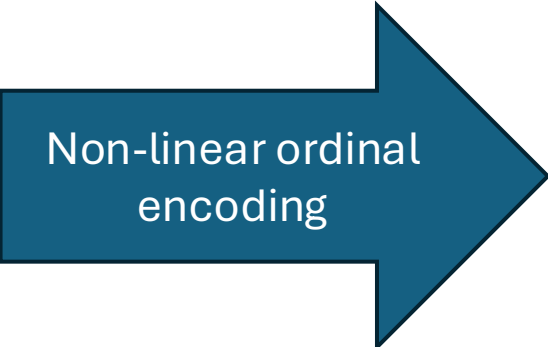
- We can replace many titles with a more common name or classify them as « Rare ».

Dependencies with Class

	Pclass	Survived
0	1	0.629630
1	2	0.472826
2	3	0.242363

Dependencies with Age

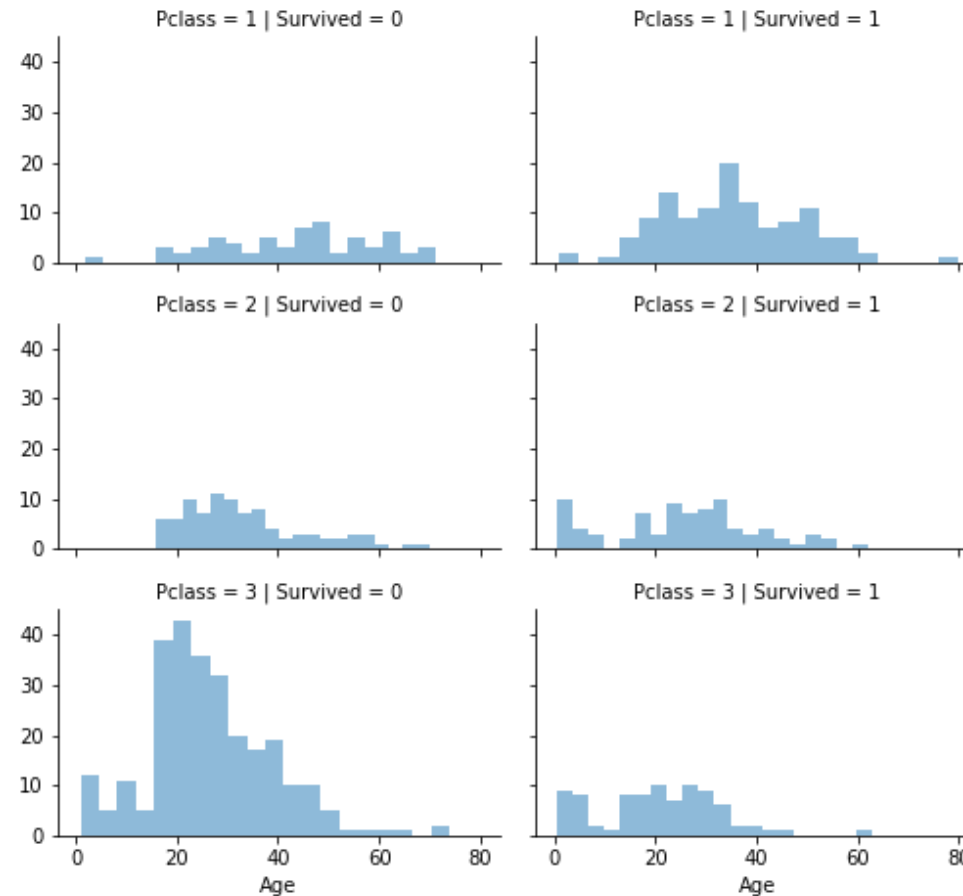
	AgeBand	Survived
0	(-0.08, 16.0]	0.550000
1	(16.0, 32.0]	0.337374
2	(32.0, 48.0]	0.412037
3	(48.0, 64.0]	0.434783
4	(64.0, 80.0]	0.090909



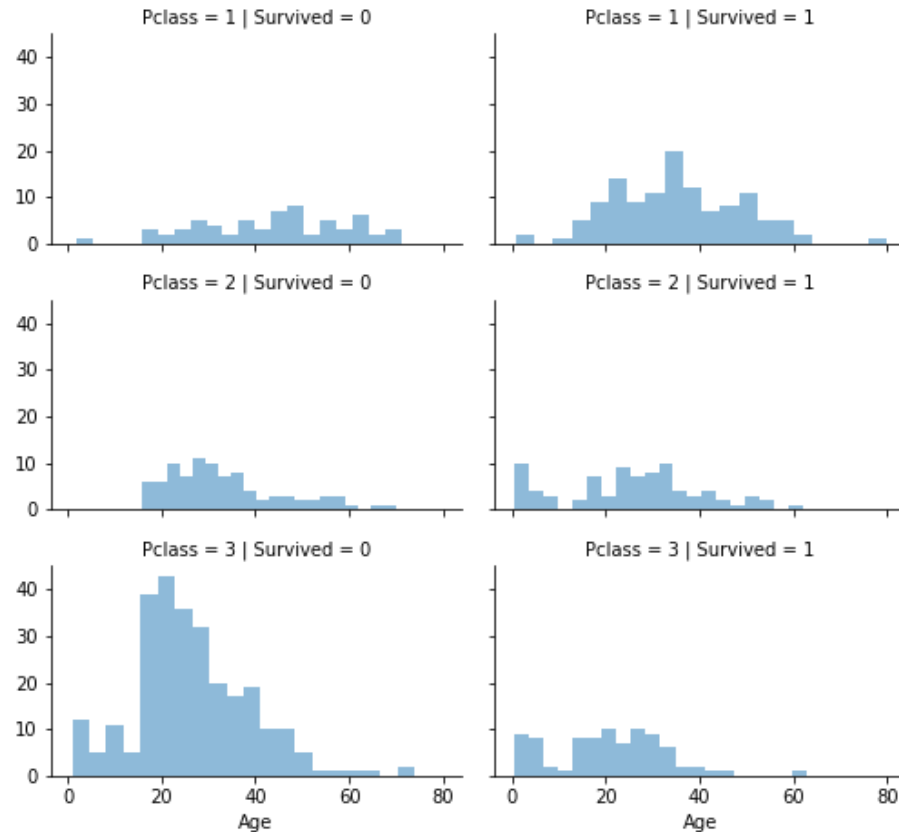
Non-linear ordinal
encoding

	Age	Survived
0	0	0.550000
1	1	0.337374
2	2	0.412037
3	3	0.434783
4	65	0.000000
5	66	0.000000
6	70	0.000000
7	71	0.000000
8	74	0.000000
9	80	1.000000

Age knowing Class and Survived



Combine features



	Age	Pclass	Survived	
0	0	1	0.888889	
1	0	2	0.904762	
2	0	3	0.400000	
3	1	1	0.728814	
4	1	2	0.422680	
5	1	3	0.244838	
6	2	1	0.616162	
7	2	2	0.446809	
8	2	3	0.100000	
9	3	1	0.534884	
10	3	2	0.352941	
11	3	3	0.111111	
12	65	1	0.000000	
13	65	3	0.000000	
14	66	2	0.000000	
15	70	1	0.000000	
16	70	2	0.000000	
17	70	3	0.000000	
18	71	1	0.000000	
19	74	3	0.000000	
20	80	1	1.000000	

- We can also create an artificial feature combining Pclass and Age as « Pclass x Age ».
- Why a product?
 - For a given age, the product increases linearly as a function of the class.
 - For a given class, the product increases linearly as the age increases.

Interpretation

- Why interactions can help? The effect of age is not the same for every passenger class!
- The table shows the survival rate as a function of passenger class (Pclass) and age (Age).

- A model with Pclass and Age only may learn

- young passengers survived often;
- older passengers survived much less often.

- A model with Pclass*Age may learn

- young passengers in 1st class and 2nd class survived very often;
- older passengers in 1st class survived sometimes
- older passengers in 3rd class survived rarely
- Pclass*Age acts like a combined “social disadvantage + age vulnerability” signal.

Passenger Class	Children (<16)	Adults (16–50)	Seniors (>50)
1st Class	92%	68%	52%
2nd Class	81%	47%	28%
3rd Class	53%	21%	8%

- So the interaction feature gives the model a way to distinguish “old rich passenger” from “old poor passenger”
- It can be interpreted as a rule like « if Pclass == 3 AND Age > 50 »

Polynomial Features

- If you have two variables (x, y) , the degree-2 polynomial features are

$$(1, x, y, x^2, xy, y^2)$$

- It works also for more variables.
- Useful to code interactions between features

Fraud Detection Example

- Suppose a bank wants to detect suspicious credit card activity.

- For each transaction, we compute two features:
 - how unusual the transaction amount is (x_1)
 - how unusual the transaction frequency is (x_2)
 - Both are normalized scores between 0 and 10.

Transaction	Amount (x_1)	Frequency (x_2)	New feature ($z = x_1^2 + x_2^2$)	Fraud?
A	1	1	2	No
B	2	3	13	No
C	7	1	50	No
D	1	8	65	No
E	6	7	85	Yes
F	8	8	128	Yes

- Notice something important:
 - a high amount alone is not enough
 - a high frequency alone is not enough
 - fraud appears when the combined abnormality becomes large
- We define a new feature:
 - $z = x_1^2 + x_2^2$
 - This measures the overall “energy” or combined anomaly level.

Feature selection

- Removing features with low variance
- Brute force (if the number of features is small)
- Existing algorithms as in

https://scikit-learn.org/stable/modules/feature_selection.html

Trade-off in Modelling

- Modelling for Prediction
- Modelling for Explanation

Modelling for Prediction

- The model's performance is more important than understanding the mechanism of data generation.
- Here, robust prediction of an outcome or reliable discrimination between different types of data is more important than explaining the underlying phenomenon.
- These data science tasks crop up often in industry, for example it may be more important to accurately predict film preferences from a set of input variables than understand how your input variables translate to film preferences.

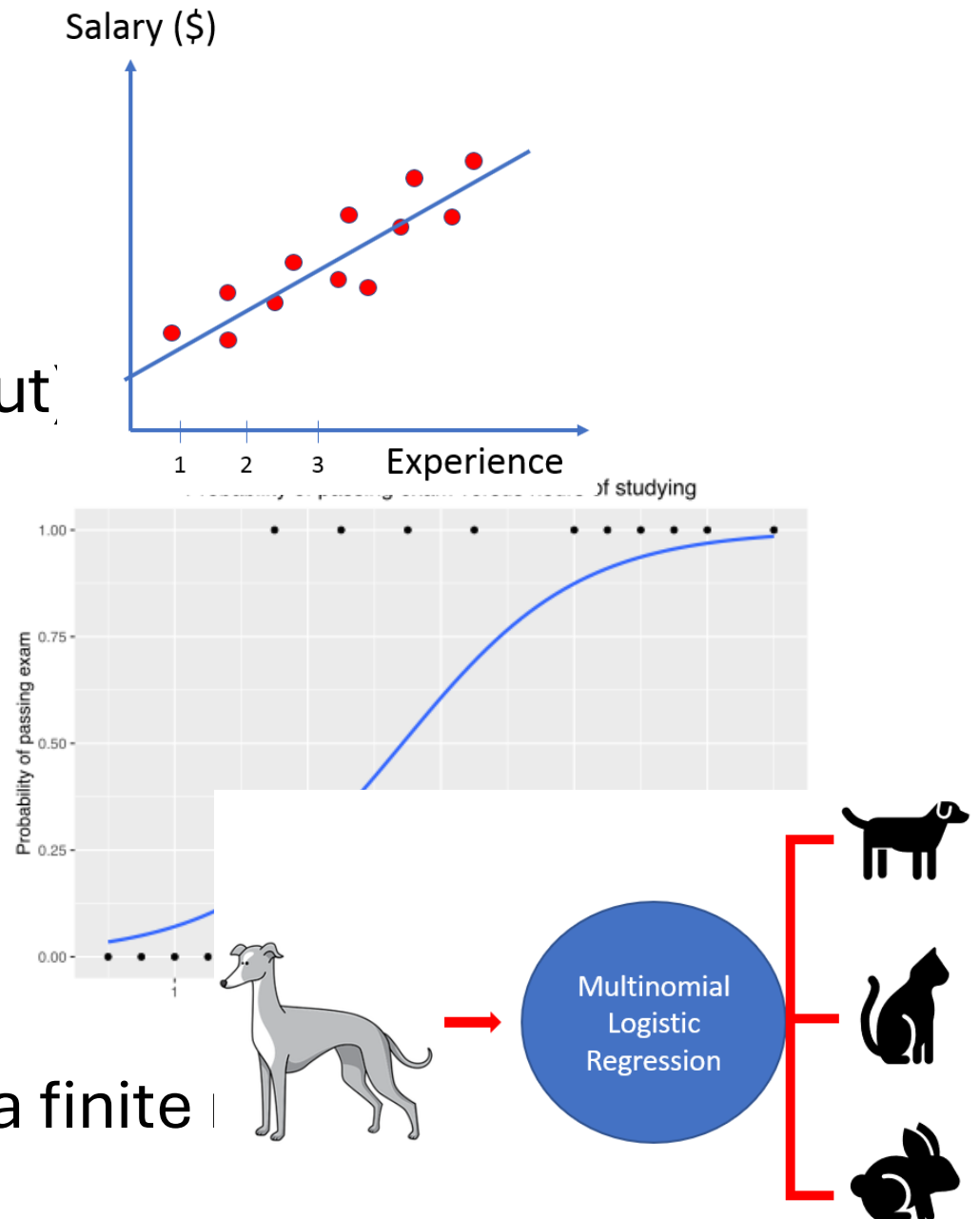
Modelling for Explanation

- A premium is placed on understanding the mechanism of data generation.
- We don't want a highly complex model that achieves excellent performance yet does not allow us to better understand the underlying phenomenon.
- When model interpretability is important there is a trade-off between model complexity and scientific value.

Linear Regression

Which regression?

- Linear regression (continuous output)
- Logistic regression (binary output)
- Softmax regression
, also called multinomial regression, a finite



Fitting Regression Models

- Simple additive multiple regression model

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_k x_k + \varepsilon$$

- Model with variable dependency
 - Example : polynomial model

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_k x^k + \varepsilon$$

- Example : model with interaction terms

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_{1,2} x_1 x_2 + \beta_{1,3} x_1 x_3 + \beta_{2,3} x_2 x_3 + \varepsilon$$

Least Squares (LS) Estimation

- Assuming n random samples

$$(y_i, x_{i1}, x_{i2}, \dots, x_{ik}), i = 1, 2, \dots, n.$$

- The estimates of the parameters for the best predicting equation:

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2} + \dots + \hat{\beta}_k x_{ik}$$

is found by choosing the values: $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_k$

which minimize the expression:

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - \hat{\beta}_2 x_{i2} - \dots - \hat{\beta}_k x_{ik})^2$$

Matrix form and optimal solution

- $SSE = SSE(\hat{\beta}) = \min_{\beta \in \mathbb{R}^k} SSE(\beta)$ with $SSE(\beta) = \|Y - A\beta\|^2$ and

$$Y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, A = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1k} \\ & \vdots & & \vdots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{nk} \end{pmatrix} \text{ and } \beta = \begin{pmatrix} \beta_0 \\ \vdots \\ \beta_k \end{pmatrix}$$

- The optimal solution $\hat{\beta}$ satisfies (**normal equations**)

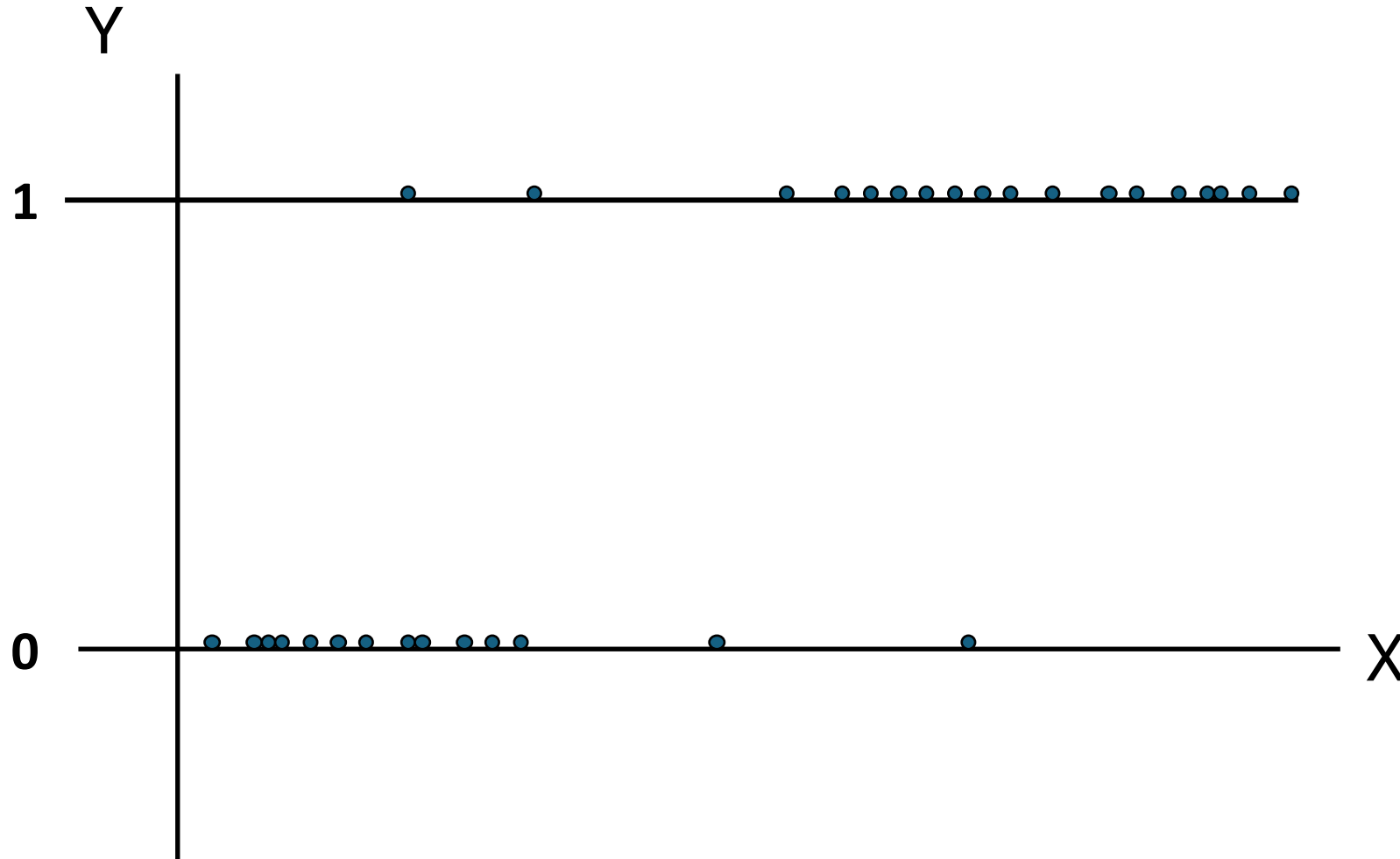
$$\nabla SSE = 2A^T A \hat{\beta} - 2A^T Y = 0.$$

- The optimal solution is then $\hat{\beta} = (A^T A)^{-1} A^T Y$.
- Of course, $A^T A$ must be invertible!

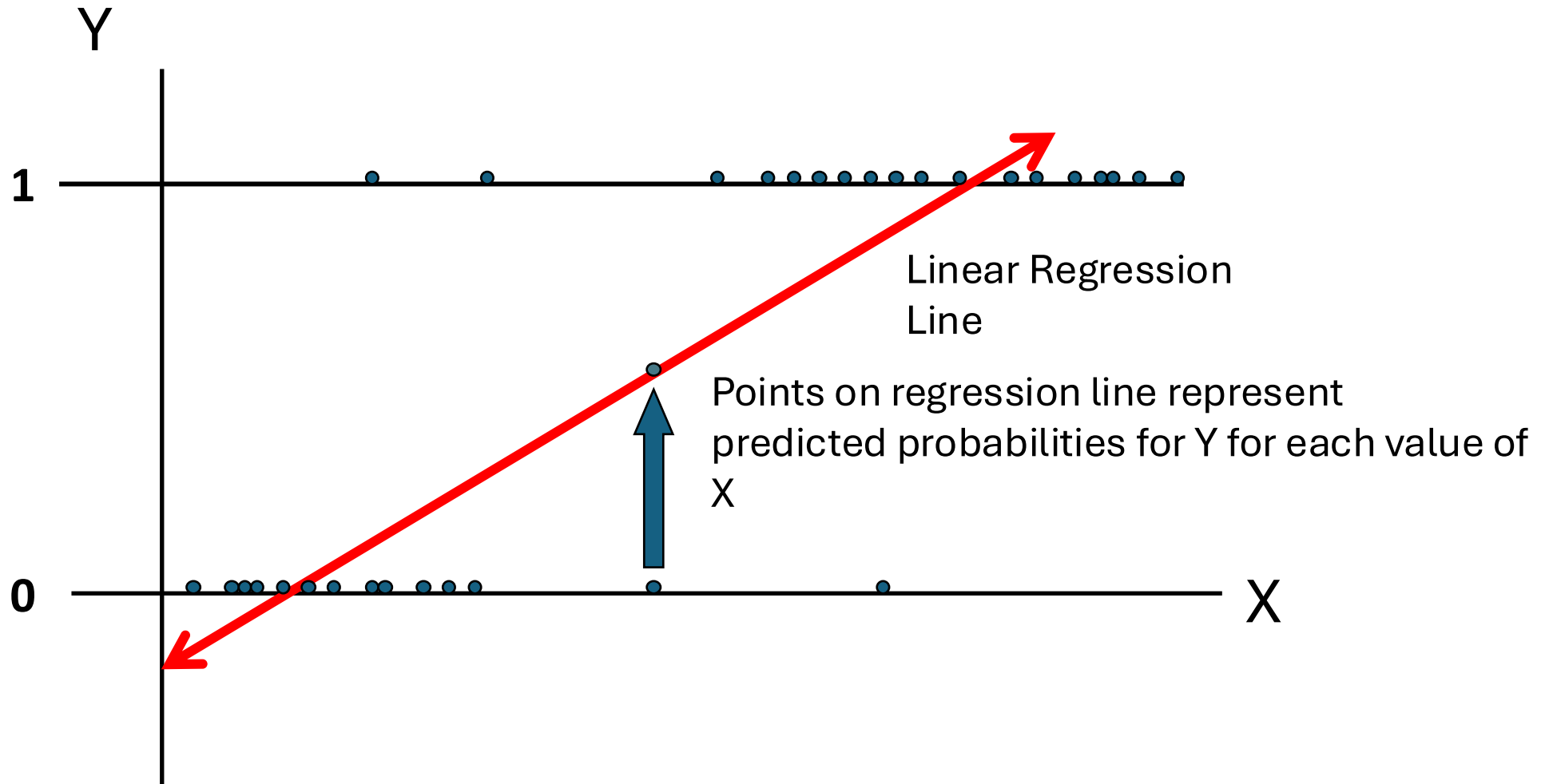
Logistic Regression

Scatterplot with $Y \in \{0,1\}$

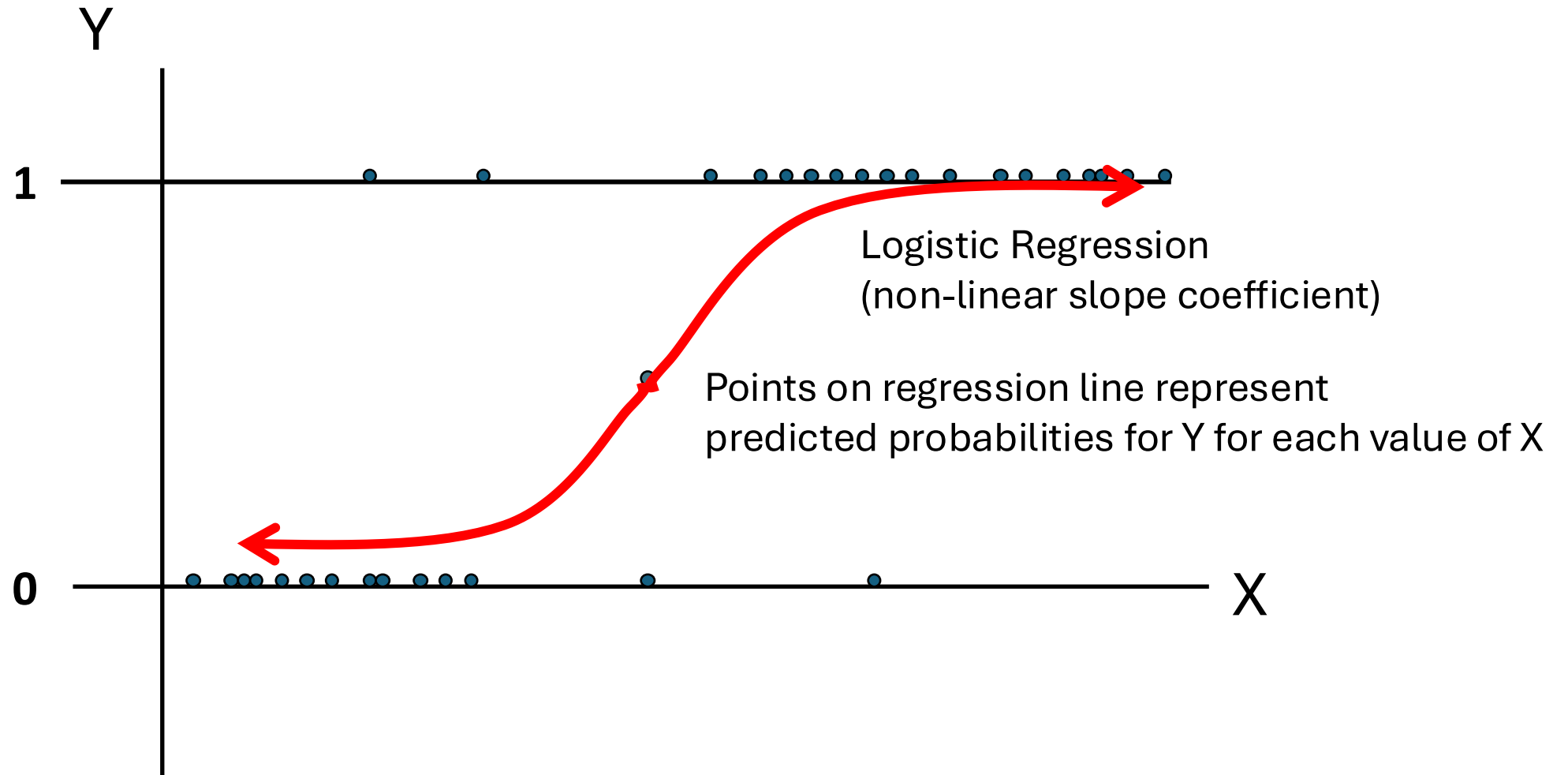
Y = Hired-Not Hired; X = Experience



Simple Linear Regression



Picture of Logistic Regression



Logistic Regression Model

- Assumption: $P = P(Y = 1|X) = P(X) = P(X_1, X_2, \dots, X_d)$
- Logistic Regression Model (LOGIT Transform):

$$\text{logit}(P(X)) = \ln \left(\frac{P(X)}{1-P(X)} \right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_d X_d$$

or

$$\text{odds}(X) = e^{\text{logit}(P(X))} = e^{\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_d X_d}$$

- The coefficients $\beta_0, \beta_1, \dots, \beta_d$ should be estimated.

Constructing the estimated model

- Training data set with N realizations of the random vector $(Y_i, X_i = (X_{i,1}, \dots, X_{i,d}))$:

$$\left\{ \left(y_1, (x_{1,1}, \dots, x_{1,d}) \right), \left(y_2, (x_{2,1}, \dots, x_{2,d}) \right), \dots, \left(y_N, (x_{N,1}, \dots, x_{N,d}) \right) \right\}$$

- The joint conditional probability of the training labels is

$$\Pr(y_1, y_2, \dots, y_N | x_1, \dots, x_N) = \Pr(Y_1 = y_1, Y_2 = y_2, \dots, Y_N = y_N | X_1 = x_1, \dots, X_N = x_N)$$

- It simplifies into

$$\Pr(y_1, y_2, \dots, y_N | x_1, \dots, x_N) = \Pr(y_1 | x_1) \Pr(y_2 | x_2) \cdots \Pr(y_N | x_N) = \prod_{i=1}^N \Pr(Y_i = y_i | X_i = x_i)$$

Assumption: independent y_i 's

Maximum Likelihood (ML)

- Recall that
 - $\ln\left(\frac{\Pr(Y=1|X)}{1-\Pr(Y=1|X)}\right) = \text{logit}(\Pr(Y = 1|X)) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_d X_d$
 - $\Pr(Y = 1|X) = f(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_d X_d) = \Pr_\beta(Y = 1|X)$ where f is the logistic function
- Hence, the joint conditional probability of the labels is

$$\Pr(y_1, y_2, \dots, y_N | x_1, \dots, x_N) = \prod_{i=1}^N \Pr(Y_i = y_i | X_i = x_i) \approx \prod_{i=1}^N \Pr_\beta(Y_i = y_i | X_i = x_i)$$

where $\beta = (\beta_0, \beta_1, \beta_2, \dots, \beta_d)$ is the vector of parameters to be estimated.

- From the ML principle, we choose parameters $\hat{\beta}$ that satisfy

$$\hat{\beta} = \underset{\beta \in \mathbb{R}^{d+1}}{\operatorname{argmax}} \prod_{i=1}^N \Pr_\beta(y_i | x_i)$$

Optimization

- The optimization is solved by a gradient-based algorithm
- For this course, just use

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

Softmax Regression

Logistic Regression with more than two outcomes

- Multinomial models for a response variable with K categories have $K - 1$ response functions.
- Linear model for each one
- It's like multivariate regression.

Multinomial Regression Model

- Assumption: $P(Y = k|X) = P_k(X) = P_k(X_1, X_2, \dots, X_d)$

- Multinomial Regression Model:

- k -th response function for $k = 1, 2, \dots, K$

$$P_k(X) = \frac{1}{Z} e^{\beta_{k,0} + \beta_{k,1}X_1 + \beta_{k,2}X_2 + \dots + \beta_{k,d}X_d} = \frac{1}{Z} e^{L_k(X)}$$

- Probability constraint:

$$P_1(X) + P_2(X) + \dots + P_K(X) = 1, \quad \forall X$$

- The constant Z is given by

$$Z = \sum_{k=1}^K e^{L_k(X)}$$

Multinomial Regression Model

- It follows that

$$P_k(X) = \frac{e^{L_k(X)}}{\sum_{k=1}^K e^{L_k(X)}} = \text{softmax}(L_1(X), L_2(X), \dots, L_K(X))$$

with

$$\text{softmax}(x_1, x_2, \dots, x_K) = \left(\frac{e^{x_1}}{\sum_{k=1}^K e^{x_k}}, \frac{e^{x_2}}{\sum_{k=1}^K e^{x_k}}, \dots, \frac{e^{x_K}}{\sum_{k=1}^K e^{x_k}} \right)$$

- The softmax function serves as the equivalent of the logistic function in binary logistic regression.
- The softmax function takes as input a tuple of K real numbers, and normalizes it into a **probability distribution** consisting of K probabilities.
- This explains why it is also called softmax regression.

Maximum Likelihood (ML)

- The likelihood function for this model is defined by

$$L = \prod_{i=1}^n \Pr_{\beta}(Y_i = y_i | X_i = x_i)$$

where $\beta = (\beta_{1,0}, \beta_{1,1}, \beta_{1,2}, \dots, \beta_{1,d}, \dots, \beta_{K,0}, \beta_{K,1}, \beta_{K,2}, \dots, \beta_{K,d}) \in \mathbb{R}^{K(d+1)}$ is the vector of parameters to be estimated.

- We get

$$L = \prod_{i=1}^n \prod_{k=1}^K P_k(x_i)^{\delta_{k,y_i}}$$

Where $\delta_{k,j} = 1$ if $k = j$ and $\delta_{k,j} = 0$ if $k \neq j$.

- The negative log-likelihood function is therefore the well-known cross-entropy:

$$-\ln L = -\sum_{i=1}^n \sum_{k=1}^K \delta_{k,y_i} \ln P_k(x_i)$$

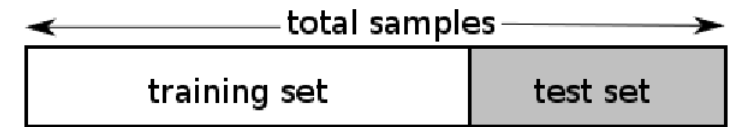
- From the ML principle, we choose parameters $\hat{\beta}$ that satisfy

$$\hat{\beta} = \underset{\beta \in \mathbb{R}^{K(d+1)}}{\operatorname{argmin}} \ln L$$

Model Validation

Holdout and Cross-Validation Methods

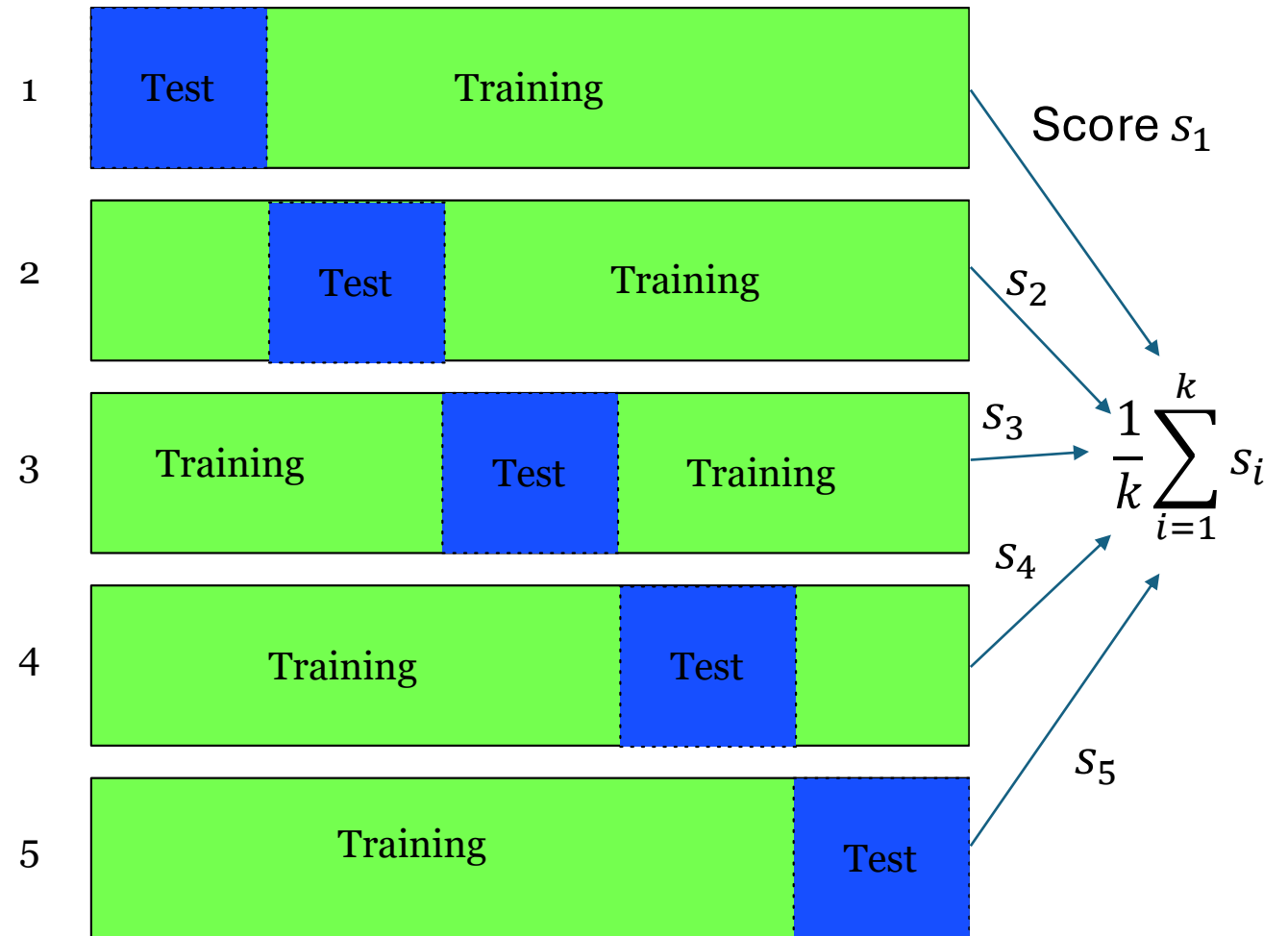
- Holdout method
 - Given data is randomly partitioned into two independent sets
 - Training set (e.g., 2/3) for model construction
 - Test set (e.g., 1/3) for accuracy estimation
 - Random sampling: a variation of holdout
 - Repeat holdout k times, accuracy = avg. of the accuracies obtained
- Cross-validation (k -fold, where $k = 10$ is most popular)
 - k -fold: randomly partition the data into k mutually exclusive subsets $D_1, \dots, D_k$, each approximately equal size. At i -th iteration, use D_i as test set and others as training set
 - Leave-one-out: k folds where $k = \#$ of tuples (the test set contains only one tuple)



k -Fold Cross-Validation

- Break up data into k folds
 - Equal positive and negative inside each fold?
- For each fold
 - Choose the fold as a temporary test set
 - Train on $k - 1$ folds, compute performance on the test fold
- Report average performance of the k runs

Iteration



Conclusion

Next

- Create your own handcrafted features based on your data analysis
- Train a linear regression
- Do not forget model validation
- Select the most significant features
- Try again if necessary